

P3570R2

optional variants in sender/receiver

Published Proposal, 2025-06-18

Author:

Fabio Fracassi ([Code University of Applied Sciences](#)) fabio@fracassi.de

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Some async interfaces for concurrent queues would be well served by using `std::optional` when used in a coroutine context. However when using sender/receiver interfaces directly the variant style visitor based on overloading has several benefits. This paper explores if and how we can get the best interface for each use case.

Table of Contents

- 1 Motivation**
- 2 Exploration**
 - 2.1 Option 1: custom awaiter
 - 2.2 Option 2: generalized value transform
 - 2.2.1 Option 2.a1: Always transform
 - 2.2.2 Option 2.a2 Opt-Out
 - 2.2.3 Option 2.b Opt-In
- 3 Proposal**
- 4 Wording**
 - 4.1 Header `<version>` synopsis [version.syn]

- 4.2 Header `<execution>` synopsis [execution.syn]
- 4.3 Execution control library [exec]
 - 4.3.1 `execution::as_awaitable` [exec.as.awaitable]

5 Implementation Experience

6 Acknowledgements

Appendix A (Workarounds)

Appendix B (Prototype)

References

Normative References

Informative References

§ 1. Motivation

This paper is a reaction to the Library Evolution(LEWG) review of the Concurrent Queues paper [P0260R13], specifically the review of revision 13. The return type of `async_pop()` has been contentious for a while, and it has been blocking consensus of an otherwise highly anticipated facility.

The contention stems from the fact that the `async_pop()` (and other senders) can be used in two different contexts. It can be used in chain of sender adaptors or can be awaited on in a coroutine. During the review it became clear that depending on the context LEWG would prefer a slightly different usage pattern. Consider the following examples (adapted from [C++ Concurrent Queues § Examples](#)) for the preferred usage:

```
auto coro_grep(stdx::coqueue<std::fs::path>& file_queue, std::string needle)
-> stdp::task<void>
{
    while (auto fname = co_await file_queue.async_pop()) {
        grep_file(*fname, needle);           // queue has data
    }

    // queue closed

    co_return;
}
```

particularly note line 4 and 5 in the coroutine example above and line 6 to 10 in the S/R example below

```
stdexec::sender auto sr_grep(auto scheduler
    , stdx::coqueue<std::fs::path>& file_queue, std::string needle)
{
    return stdexec::repeat_effect_until(
        stdexec::starts_on(scheduler
            , files.async_pop()
            | stdp::overload(
                []() -> bool { return true; },           // queue closed
                [needle](std::fs::path const& fname) -> bool { // queue has data
                    return grep_file(fname, needle);
                }
            )
        ));
}
```

We can make either of these interfaces work, depending on the completion signatures that the sender that is returned from `async_pop()` supports. To support the ideal coroutine interface the `async-pop-sender` should have the completion signature `set_value_t(std::optional<T>)`, to support the ideal S/R interface the completion signatures would need to be `set_value_t()` and `set_value_t(T)`. Whichever we support slightly penalizes the interface for the other, see [Appendix A \(Workarounds\)](#).

With this paper we want to explore if and how we could support the ideal interface for both contexts.

§ 2. Exploration

To understand a path toward a solution we need to take a short look into how senders/receivers interact with coroutines.

The sender is not directly used in the coroutine, but is transformed into an `awaitable`. The transformation is initiated by the coroutine task type (as proposed in [\[P3552R0\]](#)) which uses S/Rs `std::as_awaitable` to do the actual transformation mechanics.

§ 2.1. Option 1: custom awaiter

The simplest option is to exploit that `std::as_awaitable` provides a customization point, allowing a sender to provide its own custom `awaiter` type.

This is straightforward, and allows us to get the ideal interface for both contexts. It involves very little machinery, and in fact even bypasses most of the S/R machinery, most likely allowing for good compile times and runtime efficiency.

However the custom awaiter will only be used if the `async-pop-sender` is used directly. The result of `async_pop` can not be adapted by any sender adaptors. This will severely limit implementation freedom, and would probably cause some complicated code duplication, e.g. in the coroutine task type.

it would also make the abstraction brittle for the user, since the preferred interface would only work on unadapted senders.

§ 2.2. Option 2: generalized value transform

The second option is to hook into the `std::as_awaitable` machinery in a more generic fashion.

Currently the C++ Working draft spells out several potential transformations in `[exec.as.awaitable]`, that are tried one after the other. The one that allows senders with a single value completion signature to be used seamlessly is described in §7.3 which consist of two exposition only facilities, an `awaitable-sender` concept and a `sender-awaitable` type.

We could add another potential transformation right after that single sender step to `std::as_awaitable`. This step would take an exposition only type trait `completion-adaptor` that provides a sender adaptor (for the motivating usecase from `[P0260R13]` that adaptor would be `into_optional` as described in `[P3552R0]` Sec 4.4). `as_awaitable` would now check if the original sender that has been adapted with the transformation provided by the `completion-adaptor` trait conforms to the `awaitable-sender` concept in which case it would provide the `sender-awaitable` for the adapted sender.

The `into_optional` transform takes a sender with a `set_value_t()` and `set_value_t(T)` and changes that into a `set_value_t(std::optional<T>)` completion signature.

There are three options for how we could use the `completion-adaptor` trait:

§ 2.2.1. Option 2.a1: Always transform

The type trait is exposition only and defaults to `into_optional`. No specialisations are provided. `as_awaitable` will always try the `into_optional` transformation.

This means all senders that have compatible value completion signatures will automatically be usable in coroutines with the `std::optional<T>` return type. Users won't be able to customize the behavior. We would have to fully specify `into_optional`

§ 2.2.2. Option 2.a2 Opt-Out

Same as § 2.2.1 Option 2.a1: Always transform, but with a opt-out mechanism.

The type trait is exposition only and defaults to `into_optional`. A specialisation is provided, that will query the sender type with a new environment property `get_await_completion_adaptor`. The (original) sender can provide such a property like this:

```
struct env { auto query(get_await_completion_adaptor_t) const -> std::false_type;
auto get_env() const noexcept -> env { return {}; } }
```

In this case the trait specialisation will return a special type that `std::awaitable` uses to skip the adaption step.

This means all senders that have compatible value completion signatures will automatically be usable in coroutines with the `std::optional<T>` return type, but users could opt out in case the optional semantic do not make sense for a certain sender. We would have to fully specify `into_optional`

§ 2.2.3. Option 2.b Opt-In

Same as § 2.2.1 Option 2.a1: Always transform, but only if the sender explicitly opts-in.

The type trait is exposition only and defaults to an exposition only type that will make `std::as_awaitable` skip the adaption step (i.e. same the behavior as the current status quo). A specialisation is provided, that will query the sender type with a new environment property `get_await_completion_adaptor`, that will return the adaptor type from the query.

The (original) sender can provide such a property like this:

```
struct env { auto query(get_await_completion_adaptor_t) const -> into_optional;
auto get_env() const noexcept -> env { return {}; } }
```

Note that in this approach the senders environment property provides a the adapter type directly. This gives users full control over which adaption they want, so the mechanism would not be restricted to a

specific transformation.

This means that senders that have compatible value completion signatures will not automatically be usable in coroutines with the `std::optional<T>` return type, but sender providers could opt in when the optional semantic does make sense for a certain sender, as for example the motivating `async-pop-sender`.

§ 3. Proposal

A custom awaiter as described in § 2.1 [Option 1: custom awaiter](#) solves our immediate problem, but does so only for the specific case, and seems overly brittle.

§ 2.2.1 [Option 2.a1: Always transform](#) solves our problem in a generic fashion, but might cause new problems, because it automatically changes signatures of senders, that might be ill suited for this transformation. The same is true for § 2.2.2 [Option 2.a2 Opt-Out](#), but it would give such ill suited senders a way out.

We propose to go with § 2.2.3 [Option 2.b Opt-In](#). There is no clear evidence how often the `std::optional<T>` semantics are inappropriate for senders that otherwise fulfill the completion signatures, so we propose to go with the conservative option that will allow senders to opt-into the semantics when appropriate, but never automatically.

This Option also has the benefit of being fully generic, with very localized changes, and no need define the actual adaption type as part of either this paper or the lazy task type. Concurrent queues could either use a generic `into_optional` type or a custom adaptor for that purpose. This also leaves the design space open for the concurrent queues to provide different adaptors such as e.g. `disengage_on_close` or `disengage_on_empty`.

§ 4. Wording

§ 4.1. Header `<version>` synopsis [version.syn]

Bump the value of the existing feature test macro `__cpp_lib_senders`.

```
#define __cpp_lib_senders <bump value> // also in <execution>
```

§ 4.2. Header `<execution>` synopsis [execution.syn]

Add the declarations of `get_await_completion_adaptor_t` and `get_await_completion_adaptor` to the `<execution>` synopsis.

```
struct get_domain_t { unspecified };
struct get_scheduler_t { unspecified };
struct get_delegation_scheduler_t { unspecified };
struct get_forward_progress_guarantee_t { unspecified };
template<class CP0>
    struct get_completion_scheduler_t { unspecified };
struct get_await_completion_adaptor_t { unspecified };

inline constexpr get_domain_t get_domain{};
inline constexpr get_scheduler_t get_scheduler{};
inline constexpr get_delegation_scheduler_t get_delegation_scheduler{};
enum class forward_progress_guarantee;
inline constexpr get_forward_progress_guarantee_t get_forward_progress_guarantee{};
template<class CP0>
    constexpr get_completion_scheduler_t<CP0> get_completion_scheduler{};
inline constexpr get_await_completion_adaptor_t get_await_completion_adaptor{};
```

§ 4.3. Execution control library [exec]

Insert new subsection in [exec.queries] after `exec.get.compl.sched`:

3.5.10 execution::get_await_completion_adaptor [exec.get.await.adapt]

1. `get_await_completion_adaptor` asks a queryable object for its associated awaitable completion adaptor.
2. The name `get_await_completion_adaptor` denotes a query object. For a subexpression `env`, `get_await_completion_adaptor(env)` is expression-equivalent to `MANDATE-
NOTHROW(AS-CONST(env).query(get_await_completion_adaptor))`.
3. `forwarding_query(execution::get_await_completion_adaptor)` is a core constant expression and has value `true`.

§ 4.3.1. `execution::as_awaitable` [`exec.as.awaitable`]

in [`exec.as.awaitable`] (33.13.1) in §1 immediately after `awaitable-sender` add the following exposition only concept:

```
template<class Sndr>
concept has_queryable_await_completion_adaptor // exposition only
    = sender<Sndr>
    && requires(Sndr&& sender) {
    get_await_completion_adaptor(get_env(sender));
};
```

Change [`exec.as.awaitable`] (33.13.1).§7, inserting a new item in between (7.2) and (7.3) as follows:

(7.2) Otherwise, `(void(p), expr)` if `is_awaitable<Expr, U>` is true, where U is an unspecified class type that is not `Promise` and that lacks a member named `await_transform`.

Preconditions: `is_awaitable<Expr, Promise>` is true and the expression `co_await expr` in a coroutine with promise type U is expression-equivalent to the same expression in a coroutine with promise type `Promise`.

(7.3) Otherwise, `sender_awaitable{adapted-expr, p}` if `has_queryable_await_completion_adaptor<Expr>` and `awaitable_sender<decltype((adapted-expr)), Promise>` are both satisfied, where `adapted-expr` is `get_await_completion_adaptor(get_env(expr))(expr)`, except that `expr` is evaluated only once.

(7.4) Otherwise, `sender_awaitable{expr, p}` if `awaitable_sender<Expr, Promise>` is true.

(7.5) Otherwise, `(void(p), expr)`.

§ 5. Implementation Experience

§2.1 Option 1: custom awaiter, §2.2.1 Option 2.a1: Always transform and §2.2.3 Option 2.b Opt-In have been prototyped on top of [exec26]. The implementation is straightforward, see [Appendix B \(Prototype\)](#).

§ 6. Acknowledgements

Dietmar Kühl, for his implementation of senders/receivers, and for talking through some of the initial ideas. Ian Petersen, for helping me with the opt-in mechanism.

§ Appendix A (Workarounds)

Workaround for the coroutine case if `async-pop-sender` supports `set_value_t()` and `set_value_t(T)`

```
while (auto fname = co_await file_queue.async_pop() | stdp::into_optional(  
    grep_file(*fname, needle);  
) {
```

The drawback here is that everyone needs to remember the additional `| stdp::into_optional()` for what is most likely a very common application programming usecase.

The code for sender/receiver if `async-pop-sender` supports `set_value_t(std::optional<T>)`

```
files.async_pop() | stdp::overload(  
    [needle](std::optional<std::fs::path const>& fname) -> bool {  
        if (!fname) { return false; }  
  
        return grep_file(*fname, needle);  
    });
```

This has two drawbacks, it introduces the potential for UB, since users could forget to check the optionals engaged state. The wrapping and unwrapping of the optional could also possibly introduce some overhead.

Concievably we could also introduce a special `overload` function for this case, which would get rid of the first drawback.

```
files.async_pop()  
| stdp::overload_optional(  
    []() -> bool { return true; },
```

```
[needle](std::fs::path const& fname) -> bool {
    return grep_file(fname, needle);
});
```

§ Appendix B (Prototype)

Prototype implementation based on [\[exec26\]](#):

```
#include <beman/execution26/execution.hpp>
#include <beman/execution26/detail/queryable.hpp>
#include <optional>
#include <print>

namespace stdexec = beman::execution26;

//-----
// try out different options

//#define OPTION1(...) __VA_ARGS__
#define OPTION1(...)

#define OPTION2b(...) __VA_ARGS__
//#define OPTION2b(...)

//-----
// Optional Sender adaptor (taken from P3552-task branch)
template <typename...> struct type_list {};

inline constexpr struct into_optional_t : beman::execution26::sender_adaptor {
    template <stdexec::sender Upstream>
    struct sender {
        using upstream_t = std::remove_cvref_t<Upstream>;
        using sender_concept = stdexec::sender_t;
        upstream_t upstream;

        template <typename T> static auto find_type(type_list<type_list<T>>);
        template <typename T> static auto find_type(type_list<type_list<T>>);
        template <typename T> static auto find_type(type_list<type_list<>>);

        template <typename Env> auto get_type(Env&&) const {
```

```

        return decltype(find_type(stdexec::value_types_of_t<Upstream, ...>(),
    }
template <typename... E, typename... S>
constexpr auto make_signatures(auto&& env, type_list<E...>, type_list<S...>)
    return stdexec::completion_signatures<
        stdexec::set_value_t(decltype(this->get_type(env))),
        stdexec::set_error_t(E)...,
        S...
    >();
}

template<typename Env>
auto get_completion_signatures(Env&& env) const {
    return make_signatures(env,
        stdexec::error_types_of_t<Upstream, std::conditional_t<
            stdexec::sends_stopped<Upstream, std::is_lvalue_reference_t<Upstream>,
            type_list<stdexec::set_stopped_t()>, type_list<>>{}
        >(),
    );
}

template <typename Receiver>
auto connect(Receiver&& receiver) && {
    return stdexec::connect(
        stdexec::then(std::move(this->upstream),
            [<typename...A>(A&&... a)->decltype(get_type(stdexec::connect(receiver,
                if constexpr (sizeof...(A) == 0u)
                    return {};
                else
                    return {std::forward<A>(a)...};
            })),
        std::forward<Receiver>(receiver)
    );
}

};

template <typename Upstream>
auto operator()(Upstream&& upstream) const -> sender<Upstream> { return
} into_optional{};

namespace P3570_detail {
//-----

```

```

// this type would become part of the standard
struct get_await_completion_adaptor_t {
    template<typename T>
    auto operator()(T const& env) const noexcept
        requires requires(T&& t) { { t.query(std::declval<get_await_comple
    {
        return env.query(*this);
    }
};

inline constexpr get_await_completion_adaptor_t get_await_completion_adaptor_t{}
//-----
// implementation detail, can needs to be cleaned up
struct no_transform_t{ // implementation only tag type, no need to standardize
    // member is not necessary, only there to
    template <typename Expr> auto operator()(Expr&& expr) const { return :
};

template<typename Sender> concept _has_env = requires(Sender&& sender) {
    { stdexec::get_env(sender) };
};

template<typename Sender, typename Env = Sender>
concept has_p3570_opt_in = ::beman::execution26::sender<Sender>
    && _has_env<Sender>
    && requires(Sender&& sender, Env&& env) {
        {
            P3570_detail::get_await_completion_adaptor_t{}
        }
    };

// Exposition only trait type mentioned in the paper
template<typename Sender> struct _transform_trait { using type = no_transform_t; };
template<typename Sender> requires has_p3570_opt_in<Sender>
struct _transform_trait<Sender> {
    using type = decltype(P3570_detail::get_await_completion_adaptor(stdexec::get_env));
};

template<typename Sender> using _transform_trait_t = typename _transform_trait<Sender>::type;

// I used an additional parameter to implement the transformation, but that
// the trait could just be used internally as well.
struct as_awaitable_t {
    template <typename Expr, typename Promise, typename Transform = _transform_trait_t<Expr>
    auto operator()(Expr&& expr, Promise& promise, Transform transform = Transform{}
        if constexpr (requires { ::std::forward<Expr>(expr).as_awaitable(promise); })

```

```

        static_assert(
            ::beman::execution26::detail::is_awaitable<decltype(::std::
                Promise>,
            "as_awaitable must return an awaitable");
        return ::std::forward<Expr>(expr).as_awaitable(promise);
    } else if constexpr (::beman::execution26::detail::
        is_awaitable<Expr, ::beman::execution26::
            not ::beman::execution26::detail::awaitable_send
        using TExpr = ::std::remove_cvref_t<decltype(transform(::std::
        if constexpr ( std::is_same_v<std::remove_cvref_t<Transform>,
            || not ::beman::execution26::detail::awaitable_send
        return ::std::forward<Expr>(expr);
    } else {
        return ::beman::execution26::detail::sender_awaitable<TExpr
    }
} else {
    return ::beman::execution26::detail::sender_awaitable<Expr, Pro
}
}
};
inline constexpr ::P3570_detail::as_awaitable_t as_awaitable{};
}

//-----
// Fake concurrent queue to illustrate the mechanism

template<typename T_>
struct coqueue{
    using T = int; // I just want to demonstrate the S/R and Coroutine mech
        // and make the fake queue as simple as possible

    struct pop_sender {
        using sender_concept = stdexec::sender_t;
        using completion_signatures = stdexec::completion_signatures< stdex
            , stdex
            >;

        OPTION1(
            // Option 1: custom awaiter for pop_sender, works in the simple cas
            //         (if somefuturestd::task does not adapt the sender)
            //         + No need to change any S/R machinery, works out of th
            //         - any Sender adaption will break the special casing.
            //         - some complexity from implementing S/R operation stat

```

```

//          duplicated in awaiter/await_resume

template<typename U, typename promise_t>
struct awaiter {
    auto await_ready() -> bool { return {}; }
    auto await_suspend(std::coroutine_handle<> p) -> std::coroutine_handle<>
    auto await_resume() -> std::optional<U> {
        return queue_.empty() ? std::optional<U>{}
                               : std::optional<U>{queue_.internal_ptr()};
    }

    promise_t promise_;
    coqueue<U>& queue_;
};

template<typename promise_t>
auto as_awaitable(promise_t&& promise) {
    std::println("Using option 1");
    return awaiter<T, promise_t>{std::forward<promise_t>(promise),
    }
} // /OPTION1

// this is the S/R mechanism for the "queue" (slideware version),
// Option 1 bypasses this part of the machinery in the coroutine case
template<stdexec::receiver rcvr>
struct state {
    using _result_t = T;
    using _complete_t = void (state&);

    rcvr rcvr_;
    _complete_t* complete_;
    coqueue<T>& queue_;

    explicit state(rcvr&& r, coqueue<T>& queue)
        : rcvr_{std::move(r)}
        , complete_{[](state& self) {
            // here be synchronization
            if ( not self.queue_.empty() ) { // this would check for empty queue
                stdexec::set_value(std::move(self.rcvr_), self.queue_.internal_ptr());
            } else {
                stdexec::set_value(std::move(self.rcvr_));
            }
        }}
};

```

```

        , queue_{queue}
    {
        std::println("Using option 2 or S/R directly");
    }

    using operation_state_concept = stdexec::operation_state_t;
    auto start() noexcept -> void {
        (*complete_)(*this);
    }

// Option 2:
// this needs modification in stdexec::as_awaitable, so that it can
// should result in an awaiter that returns `std::optional`s.
// In the beman::execution26 impl this would mean one additional exp

// Option 2.a1
//     any sender that has a completion signature of `stdexec::set_v
//     is automatically considered an awaiter that returns `std::op
// Option 2.a2
//     same as 2.a1, plus a mechanism to opt out (via environments o
// Option 2.b
//     senders have to explicitly opt in. (via environments or a typ

};

// Opt-In (Option 2.b)
OPTION2b(
    struct env { auto query(P3570_detail::get_await_completion_adaptor
    auto get_env() const noexcept -> env { return {}; }
    ) // /OPTION2b

// common impl
template<stdexec::receiver rcvr> auto connect(rcvr r) { return sta

coqueue<T>& queue_;
};

auto async_pop() -> pop_sender { return pop_sender{*this}; }

// Fake queue – returns count ints in decending order queue turns empty
auto empty() -> bool { return count_ == 0; }

```

```

    auto internal_pop_() -> T { return count_--; }
    int count_;
};

//-----
// Task type, ducttape version (P3552 lazy)
struct task{
    struct promise_type {
        auto initial_suspend()                -> std::suspend_never
        auto final_suspend() noexcept        -> std::suspend_always
        [[noreturn]] auto unhandled_exception() -> void
        auto unhandled_stopped()              -> std::coroutine_handle<>

        auto get_return_object()               -> task
        auto return_void()                     -> void

        template <stdexec::sender Awaitable>
        auto await_transform(Awaitable&& awaitable) noexcept -> decltype(auto) {
            return P3570_detail::as_awaitable(std::forward<Awaitable>(awaitable));
        }

    private:
        auto tag() -> char const* { return ""; }
    };
};

//-----
// Example -> usage code in the coroutine

auto use(int i) -> void { std::println("used #{i}", i); }

auto coro(coqueue<int>& data) -> task {

    while(auto item = co_await data.async_pop()) {
        use(*item);
    }

    co_return;
}

auto main() -> int {
    coqueue<int> queue{3};

```



```
auto _ = coro(queue);
std::println(" ---[done]----- ");

return EXIT_SUCCESS;
}
```

§ References

§ Normative References

[EXEC.AS.AWAITABLE]

[exec.as.awaitable]. 2025-02-06. URL: <https://eel.is/c++draft/exec.as.awaitable>

[P0260R13]

Lawrence Crowl; et al. *C++ Concurrent Queues*. 2024-12-10. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p0260r13.html>

§ Informative References

[EXEC26]

Dietmar Kühl. *beman.execution: Building Block For Asynchronous Programs*. 2025-01-25. URL: <https://github.com/bemanproject/execution>

[P3552R0]

Dietmar Kühl; Maikel Nadolski. *Add a Coroutine Lazy Type*. 2025-01-13. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3552r0.pdf>